R3.05 · TD-TP 07

PROGRAMMATION SYSTÈME

Les tubes

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Les agents du LAMAE ne se parlent plus par fichiers : trop de traces. Ils ont monté une radio clandestine faite de tubes entre processus. Ta mission : construire les canaux, déjouer le brouillage et établir un protocole que personne ne pourra falsifier.

OBJECTIFS DE LA SÉANCE

- ☐ Créer un tube avec `pipe` **avant** `fork` et identifier `p[0]` (lecture) et `p[1]` (écriture).
- ☐ Expliquer le comportement de `read` et `write` sur un tube : blocage, fin de fichier, `SIGPIPE`.
- ☐ Fermer systématiquement les descripteurs inutiles pour éviter les interblocages.
- ☐ Définir et **valider** un protocole de messages entre plusieurs écrivains et un lecteur.
- ☐ Rediriger l'entrée ou la sortie standard avec `dup2` (reproduire `ls -l | wc -l`).
- ☐ Créer un tube nommé avec `mkfifo` et des droits restreints.

FIL ROUGE Opération « Radio clandestine »

Chaque mission rapporte un **drapeau** `FLAG{...}` à noter dans `badge07.txt`. Exercices cotés de ★ (incontournable) à ★★★ (pour aller loin). Rappel des règles de la radio : un canal qu'on n'utilise pas se **ferme**, un message reçu se **vérifie**, une valeur de retour se **teste**.

Mission	Contenu	Durée
0 · Le matériel	`pipe`, `read`, `write` sur un tube (lecture du cours)	10 min
1 · Premier contact	Père → fils, lecture partielle	15 min
2 · Le relais	Deux fils qui communiquent	15 min
3 · Le brouillage	Deux écrivains, un lecteur : chasse aux bogues	20 min
4 · Le protocole	Messages de taille fixe, PID, validation	30 min
5 · Le tuyau du shell	`dup2` : `ls -l | wc -l` en C	20 min
6 · La boîte aux lettres	Tube nommé `mkfifo`	15 min
Bonus	Pour les rapides : capacité du tube, `SIGPIPE`, ping-pong	—

COMPILATION

Les tubes, `fork`, `dup2` et `mkfifo` relèvent de la norme **POSIX**, pas du C standard. Avec `-std=c17`, placez `#define _POSIX_C_SOURCE 200809L` en **première ligne** de chaque programme, avant tout `#include`.

```
• • • TERMINAL
$ gcc -std=c17 -Wall -Wextra -pedantic -g radio.c -o radio
```

Mission 0 — Le matériel : pipe, read, write

L'objectif de ce TP est d'utiliser la primitive `pipe` pour transférer des données entre processus **ayant un lien de parenté** (créés par `fork`). Un tube est un tampon du noyau, en FIFO, avec deux extrémités ; il n'a pas de nom dans l'arborescence (voir CM 03b).

Création : pipe()

```c
#include <unistd.h>
int pipe(int p[2]);      // 0 si succès, -1 si erreur (trop de descripteurs ouverts...)
```

`pipe` remplit le tableau `p` : `p[0]` est le descripteur pour **lire** (sortie du tube), `p[1]` celui pour **écrire** (entrée du tube).

```c
int p[2];
if (pipe(p) == -1) {
    perror("pipe");
    exit(EXIT_FAILURE);
}
```

Le tube doit être créé **avant** le `fork` : le fils hérite alors des deux descripteurs. Il n'y a pas d'`open` pour un tube anonyme (il n'a pas de nom) ; en revanche on le ferme avec `close`.

Lecture : read()

```c
char buf[100];
close(p[1]);                        // le lecteur ferme l'extrémité d'écriture
ssize_t nbLu = read(p[0], buf, 20);  // demande AU PLUS 20 octets
```

Les octets lus sont rangés dans `buf` ; leur nombre est la valeur de retour. Attention : `read` ne place **pas** de `'\0'` final. Fermer le descripteur inutile `p[1]` n'est pas une simple politesse : c'est ce qui permet de détecter la fin de fichier (voir ci-dessous).

Écriture : write()

```c
char buf[100] = "texte a ecrire";  // on initialise un tableau ; « buf = "..." » est interdit
close(p[0]);                       // l'écrivain ferme l'extrémité de lecture
write(p[1], buf, strlen(buf));     // renvoie le nombre d'octets écrits, ou -1
```

Une écriture de **au plus `PIPE_BUF` octets** (4096 sous Linux, `<limits.h>`) est **atomique** : ses octets restent consécutifs dans le tube, même avec plusieurs écrivains. Au-delà, le noyau peut découper l'écriture et entrelacer les données de plusieurs écrivains.

> **PIÈGE CLASSIQUE** **PIPE_BUF n'est pas la taille du tube**
>
> `PIPE_BUF` (4096) est la limite d'**atomicité**. La **capacité** d'un tube est plus grande : 64 Kio (65536 octets) par défaut sous Linux. Vous la mesurerez dans le bonus.

Comportement de read() et write()

Situation	Résultat
`read` : le tube contient des données	Lit ce qui est disponible, au plus le nombre demandé ; renvoie ce nombre (peut être inférieur à la demande).
`read` : tube vide, il reste au moins un écrivain (`p[1]` ouvert quelque part)	**Bloque** jusqu'à l'arrivée de données.
`read` : tube vide, **plus aucun** écrivain	Renvoie **0** : c'est la **fin de fichier**.
`read` non bloquant (`O_NONBLOCK`) sur un tube vide avec écrivains	Renvoie **-1** avec `errno == EAGAIN` (ce n'est pas une fin de fichier).
`write` : plus aucun lecteur	Le processus reçoit le signal **`SIGPIPE`** et meurt (« Broken pipe ») ; si `SIGPIPE` est ignoré, `write` renvoie -1 avec `errno == EPIPE`.
`write` : tube plein, il reste des lecteurs	**Bloque** jusqu'à ce qu'un lecteur libère de la place.
`write` non bloquant sur un tube plein	Renvoie **-1** avec `errno == EAGAIN` (ou écrit partiellement si la demande dépasse `PIPE_BUF`).

```c
int fl = fcntl(p[0], F_GETFL);           // rendre la lecture non bloquante
fcntl(p[0], F_SETFL, fl | O_NONBLOCK);   // (#include <fcntl.h>)
```

PIÈGE CLASSIQUE **L'interblocage (deadlock)**

Deux processus qui dialoguent par deux tubes `p` et `q`, et qui commencent **tous les deux** par lire :

```c
/* PROCESSUS 1 */                /* PROCESSUS 2 */
read(p[0], ch, n);               read(q[0], ch, n);
write(q[1], ch, n);              write(p[1], ch, n);
```

Les deux tubes sont vides au départ : les deux `read` bloquent pour toujours. Il faut fixer un **ordre** (qui parle en premier). Autre cause classique : un lecteur qui a oublié de fermer **son propre** `p[1]` attend une fin de fichier qui n'arrivera jamais, puisqu'il reste un écrivain... lui-même.

Mission 1 — Premier contact

Le père envoie au fils un message dans un tube ; le fils le lit caractère par caractère et l'affiche. Récupérez `code/sujet07/exemple1.c`.

exemple1.c (extrait)

```c
static const char message[] = "Cela provient d'un tube";   // 23 caractères + '\0'
...
if (pid == 0) {                 /* fils : lecteur */
    char c;
    close(p[1]);                /* sinon read ne renverra jamais 0 */
    while (read(p[0], &c, 1) > 0)
        putchar(c);
    putchar('\n');
    close(p[0]);
    exit(EXIT_SUCCESS);
}
close(p[0]);                    /* père : écrivain */
if (write(p[1], message, strlen(message)) == -1)
    perror("write");
close(p[1]);                    /* le fils verra la fin de fichier */
wait(NULL);
```

EX 1 Premier contact

★ 15 min

1. Compilez et exécutez `exemple1.c`. Repérez où se produit l'arrêt de la boucle du fils.
2. Commentez le `close(p[1])` **du fils** et relancez. Que se passe-t-il ? Pourquoi ? (Arrêtez avec Ctrl+C, puis observez la situation avec `ps -o pid,stat,cmd` lancé depuis un autre terminal.)
3. Remplacez la boucle du fils par une seule lecture de **30** octets dans un tableau `buf`, puis affichez `nbLu` et le message. Combien d'octets sont lus ? Pourquoi faut-il écrire `buf[nbLu] = '\0';` avant d'afficher `buf` avec `%s`, et donc déclarer `char buf[31]` ?

Drapeau : `FLAG{` + le nombre d'octets effectivement lus à la question 3 + `}`.

> **SÉCURITÉ** **read ne termine jamais une chaîne**
>
> Afficher avec `%s` un tampon rempli par `read` sans y avoir placé `'\0'` fait lire `printf` **au-delà** des données reçues : affichage de déchets, voire fuite d'informations ou plantage. Réservez toujours une case de plus et terminez explicitement le tampon, après avoir **testé** que `read` n'a pas renvoyé -1.

Mission 2 — Le relais : deux fils

EX 2 Deux agents en liaison

★ 15 min

Écrivez un programme où ce sont **deux fils** qui communiquent : le premier fils est écrivain (il envoie 5 lignes « Agent i au rapport »), le second est lecteur (il affiche tout ce qu'il reçoit jusqu'à la fin de fichier). Le père crée le tube, lance les deux fils, puis attend leur fin.

1. Combien de descripteurs du tube sont ouverts au total après les deux `fork` ? Lesquels chaque processus doit-il fermer ?
2. Que se passe-t-il si le **père** oublie de fermer `p[1]` ? Vérifiez.

> **INDICE**
>
> Faites un schéma des trois processus et des deux extrémités du tube, comme en CM 03b. Une extrémité n'est réellement fermée que lorsque **tous** les processus qui la détenaient l'ont fermée.

Drapeau : `FLAG{` + le nombre total de `close` sur le tube nécessaires dans les trois processus + `}`.

Mission 3 — Le brouillage : deux écrivains, un lecteur

Deux fils écrivent respectivement `"ABC...Z"` et `"abc...z"` par blocs de **3** caractères (avec une pause pour simuler d'autres traitements), et un troisième fils lit le tube par blocs de **4**. Le code est fourni dans `code/sujet07/brouillage.c`, fidèle à l'exemple de l'ancien polycopié.

brouillage.c (extrait)

```c
static char seq1[27] = "ABCDEFGHIJKILMOPQRSTUVWXYZ";   /* écrite par FILS1 */
static char seq2[27] = "abcdefghijklmnopqrstuvwxyz";   /* écrite par FILS2 */

static void f1(void)
{
    close(p[0]);
    for (int i = 0; i < 26; i += 3) {
        write(p[1], &seq1[i], 3);
        sleep(3);
    }
    close(p[1]);
}

static void f3(void)
{
    char s[5];
    close(p[1]);
    while (read(p[0], s, 4) > 0)
        printf("%s", s);
    close(p[0]);
    printf("\n");
}
```

L'ancien polycopié annonçait le résultat : `ABCabcDEFdefGHIghiJKILMOjklPQRmnoSTUpqrVWXYZstuvwxyz`.

EX 3 Chasse aux bogues

★★ 20 min

Ce programme compile sans avertissement, mais il contient **au moins quatre** défauts. Trouvez-les, expliquez-les, puis corrigez-les.

1. Comparez attentivement `seq1` à l'alphabet. Le « `JKILMO` » du résultat annoncé s'explique-t-il vraiment par l'ordonnancement ?
2. `s` est-il une chaîne valide quand on l'affiche avec `%s` ? Qu'affiche-t-on si `read` renvoie 1, 2 ou 3 ?
3. Que vaut `i` au dernier tour de la boucle de `f1` ? Combien de caractères **utiles** restent à partir de `seq1[24]` ? Qu'écrit-on alors dans le tube ?
4. Que fait le père après avoir lancé ses trois fils ? Quel est le problème (pensez au prompt du shell et aux orphelins du sujet 06) ?
5. Les valeurs de retour de `write` et de `fork`… sont-elles toutes testées ?

Une fois le programme corrigé, relancez-le plusieurs fois : l'ordre des blocs change-t-il ? Les blocs de 3 sont-ils jamais coupés ? Pourquoi ?

Drapeau : `FLAG{` + la lettre qui manque dans `seq1` + `}`.

> **CULTURE** **Atomicité et protocole**
>
> Chaque bloc de 3 octets (moins de `PIPE_BUF`) est écrit d'un seul tenant, mais les écrivains agissent indépendamment : leurs blocs s'enchaînent dans un ordre quelconque, et le lecteur, qui lit par 4, recoupe les blocs à sa façon. Sans **protocole** (taille fixe des messages, identité de l'émetteur), le lecteur ne peut pas savoir qui a écrit quoi. C'est l'objet de la mission suivante.

Mission 4 — Le protocole de la radio

Deux processus écrivains envoient des messages dans **le même** tube ; un processus lecteur les lit et les affiche **selon leur provenance** : messages du premier écrivain sur la sortie standard (`stdout`), messages du second sur la sortie d'erreur (`stderr`).

Format des messages. Chaque message fait exactement **10 octets** :

Champ	Taille	Contenu
PID de l'émetteur	6	Justifié à droite, complété par des espaces
Séparateur	1	Un espace
Message	3	Trois caractères (un indicatif comme `ABC`)

Exemple : `"  1234 ABC"` est le message `ABC` envoyé par le processus de PID 1234. Toutes les lectures et écritures se font par blocs de **10 octets exactement** ; l'encodage utilise `snprintf`, le décodage `sscanf`.

EX 4 — Établir la liaison

★★ 30 min

Complétez `code/sujet07/protocole_squelette.c` :

1. `encoder` : fabriquez le message avec `snprintf(msg, taille, "%*d %.3s", LG_PID, (int) pid, code)` et **vérifiez** que la valeur de retour vaut exactement `LG_MSG`.
2. `ecrireTout` et `lireExactement` : bouclez sur `write` et `read` tant que les 10 octets ne sont pas transférés (un `read` peut renvoyer moins que demandé).
3. `decoder` : extrayez le PID et le code avec `sscanf` (largeurs de champ bornées) et **testez** sa valeur de retour.
4. Le père crée l'écrivain 1, puis l'écrivain 2, et mémorise leurs PID ; il crée **ensuite** le lecteur, qui hérite donc des deux PID par le `fork`. Le lecteur range chaque message sur `stdout` ou `stderr` selon le PID ; tout message d'un PID inconnu ou mal formé est signalé « message suspect » et ignoré.
5. Testez en séparant les sorties : `./protocole > radio1.txt 2> radio2.txt`.

> **PIÈGE CLASSIQUE — Un PID tient-il sur 6 caractères ?**
>
> Affichez `cat /proc/sys/kernel/pid_max`. Sur beaucoup de systèmes récents (et sous WSL), il vaut **4194304** : un PID peut avoir **7 chiffres** et ne tient plus dans le champ de 6 du protocole. `snprintf` écrit alors 11 caractères, le message est décalé et le lecteur se désynchronise. Votre `encoder` doit détecter ce cas ; le squelette permet de recompiler avec `-DLG_PID=7`. Morale : un format de taille fixe se dimensionne d'après les **valeurs maximales possibles**, pas d'après les valeurs habituelles.

> **SÉCURITÉ — Ne jamais croire un message sur parole**
>
> Le lecteur ne doit faire **aucune** hypothèse sur ce qu'il reçoit : longueur exacte (message tronqué en fin de tube), retour de `sscanf` (champ non numérique), PID attendu (un autre processus du même utilisateur peut écrire dans le tube s'il en hérite), code de 3 caractères affichables. Un champ reçu n'est **jamais** une chaîne terminée : on le copie dans un tampon de taille `LG_MSG + 1` qu'on termine soi-même. C'est la même discipline que pour les sockets (sujet 09).

DÉFI **L'agent double** ★★★

Ajoutez un troisième écrivain « agent double » qui envoie des messages forgés : un faux PID (celui de l'écrivain 1), un message de 9 octets, un message contenant `%s%s%s`. Votre lecteur résiste-t-il ? Que peut-il détecter, et que ne peut-il **pas** détecter avec ce protocole ? (Indice : l'authenticité d'un émetteur ne se prouve pas avec une donnée qu'il choisit lui-même.)

POUR LES RAPIDES **Le lecteur indépendant** ★★★

L'ancien énoncé demandait de « transmettre les PID des deux écrivains au lecteur via les arguments du programme ». Rendez-le cohérent : écrivez un **programme séparé** `lecteur.c` dont l'usage est `./lecteur PID1 PID2` et qui lit les messages sur son **entrée standard**. Le lanceur crée le tube et les deux écrivains, puis crée un fils qui fait `dup2(p[0], STDIN_FILENO)` et `execl("./lecteur", "lecteur", pid1, pid2, (char *) NULL)`. Validez les arguments avec `strtol` (et non `atoi`).

Drapeau : `FLAG{` + le nombre de chiffres de `pid_max` sur votre machine + `}`.

Mission 5 — Le tuyau du shell : dup2

Quand vous tapez `ls -l | wc -l`, le shell crée un tube, puis deux fils : le premier branche sa **sortie standard** sur `p[1]` et exécute `ls`, le second branche son **entrée standard** sur `p[0]` et exécute `wc`. `ls` et `wc` ne savent même pas qu'ils parlent à un tube.

```c
#include <unistd.h>
int dup2(int ancien, int nouveau);   // fait de « nouveau » une copie de « ancien »
                                     // (ferme d'abord « nouveau » s'il était ouvert)

dup2(p[1], STDOUT_FILENO);   // désormais, écrire sur 1 = écrire dans le tube
close(p[0]);
close(p[1]);                 // les originaux ne servent plus : on les ferme
execlp("ls", "ls", "-l", (char *) NULL);
perror("execlp");            // n'est atteint que si exec a échoué
```

EX 5 Reproduire ls -l | wc -l

★★ 20 min

1. Écrivez `pipeline.c` qui reproduit exactement `ls -l | wc -l` avec `pipe`, deux `fork`, `dup2` et `execlp`. Le père ferme les deux extrémités et attend ses deux fils.
2. Comparez votre résultat avec celui du shell.
3. Oubliez volontairement un `close(p[1])` dans le père, puis dans le fils `wc`. Que se passe-t-il dans chaque cas ? Pourquoi `wc` ne se termine-t-il jamais ?
4. Avec `strace -f -e trace=pipe2,pipe,dup2,close,execve ./pipeline`, retrouvez la séquence d'appels système de chaque processus.

Drapeau : `FLAG{` + le nom de l'appel système qui duplique un descripteur sur un numéro choisi + `}`.

SÉCURITÉ **Descripteurs qui fuient**

Tout descripteur resté ouvert traverse le `fork` **et** l'`exec` : le programme lancé hérite de vos tubes, fichiers et sockets. Outre les interblocages, c'est une fuite de privilèges si le programme exécuté est moins digne de confiance. Fermez tout ce qui ne sert pas avant `exec`, ou créez les descripteurs avec l'option *close-on-exec* (`O_CLOEXEC`, `pipe2(p, O_CLOEXEC)` sous Linux).

Mission 6 — La boîte aux lettres : tube nommé

Un tube anonyme n'est accessible qu'au processus qui l'a créé et à ses descendants. Pour faire communiquer deux programmes **sans lien de parenté**, on utilise un **tube nommé** (FIFO), qui a un nom dans l'arborescence :

```
• • • TERMINAL
$ mkfifo canal
$ ls -l canal
prw-r--r-- 1 agent info 0 canal
$ cat canal &          # le lecteur attend un écrivain
$ echo "rendez-vous à minuit" > canal
rendez-vous à minuit
```

```c
• • • C
#include <sys/stat.h>
int mkfifo(const char *chemin, mode_t mode);   // droits filtrés par le umask
int fd = open(chemin, O_WRONLY);               // bloque jusqu'à l'arrivée d'un lecteur
```

EX 6 La boîte aux lettres

★★ 15 min

1. Écrivez `recepteur.c` (crée la FIFO si elle n'existe pas, l'ouvre en lecture, affiche tout ce qu'il reçoit) et `emetteur.c` (ouvre la FIFO en écriture et y recopie son entrée standard).
2. Lancez-les dans deux terminaux, dans un ordre puis dans l'autre. Lequel bloque sur `open`, et jusqu'à quand ?
3. Quelle lettre `ls -l` affiche-t-il en tête des droits d'une FIFO ?

> **SÉCURITÉ** **Une FIFO est un fichier comme un autre**
>
> Créez-la avec `mkfifo(chemin, 0600)` : seul son propriétaire peut lire et écrire (le `umask` peut encore retirer des droits, jamais en ajouter). Une FIFO créée en `0666` dans `/tmp` permet à n'importe quel utilisateur de la machine d'écouter ou d'injecter des messages. Si la FIFO existe déjà (`EEXIST`), vérifiez avec `lstat` que c'est bien une FIFO (`S_ISFIFO`), qu'elle vous appartient et qu'elle n'est pas accessible aux autres, avant de l'ouvrir : sinon un attaquant peut y avoir placé un lien symbolique ou un fichier ordinaire.

Drapeau : `FLAG{` + cette lettre + `}`.

> **POUR LES RAPIDES** **Mesures et dialogue** ★★★
>
> 1. **Capacité.** Rendez l'écriture non bloquante (`O_NONBLOCK`) et écrivez octet par octet dans un tube que personne ne lit, jusqu'à obtenir `EAGAIN`. Combien d'octets le tube contient-il ? Comparez à `PIPE_BUF`. **Drapeau bonus :** `FLAG{` + cette capacité + `}`.
> 2. **SIGPIPE.** Fermez toutes les extrémités de lecture, puis écrivez : que se passe-t-il ? Recommencez après `signal(SIGPIPE, SIG_IGN)` et affichez `errno`.
> 3. **Ping-pong.** Père et fils s'échangent un entier par **deux** tubes (le père envoie n, le fils renvoie 2n+1) dix fois de suite, sans interblocage. Qui doit parler en premier ? Que se passe-t-il si les deux commencent par `read` ?

Bilan de la séance

À RETENIR

- `pipe` **avant** `fork` ; `p[0]` pour lire, `p[1]` pour écrire.
- Chaque processus **ferme** les extrémités dont il ne se sert pas : sinon le lecteur n'obtient jamais la fin de fichier (`read` renvoie 0).
- `read` peut renvoyer **moins** que demandé et ne termine jamais la chaîne ; `write` peut échouer (`EPIPE`, `EAGAIN`).
- Écrire dans un tube sans lecteur déclenche `SIGPIPE`.
- Écritures de moins de `PIPE_BUF` octets atomiques ; capacité du tube : 64 Kio sous Linux.
- Plusieurs écrivains = un **protocole** (taille fixe, identité) et un lecteur qui **valide** chaque message.
- `dup2` + `exec` : c'est ainsi que le shell construit `ls | wc`. FIFO : `mkfifo(chemin, 0600)`.

- ☐ J'ai mes 6 drapeaux dans `badge07.txt` (missions 1 à 6).
- ☐ Aucun de mes programmes ne laisse un processus bloqué ou un zombie (`ps -o pid,stat,cmd`).
- ☐ Mes programmes compilent sans **aucun** avertissement.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 2 pipe`, `man 2 read`, `man 2 write`	Prototypes, valeurs de retour, cas d'erreur (`EPIPE`, `EAGAIN`, `EINTR`).
`man 7 pipe`	Capacité, `PIPE_BUF`, atomicité, comportement bloquant et non bloquant : la référence.
`man 2 dup2`, `man 2 fcntl`	Redirections ; `O_NONBLOCK`, `FD_CLOEXEC`.
`man 3 mkfifo`, `man 7 fifo`	Tubes nommés et règles d'ouverture.
`strace -f ./prog`	Voir les appels système de tous les processus (`-f` suit les fils).
`ls -l /proc/<pid>/fd`	Lister les descripteurs ouverts d'un processus : idéal pour traquer un `p[1]` oublié.
M. Kerrisk, *The Linux Programming Interface*, chap. 44	Tubes et FIFO en détail, avec de nombreux exemples.
A. Tanenbaum, *Systèmes d'exploitation*	Communication interprocessus, producteur/consommateur.
J.-M. Rifflet, *La programmation sous UNIX*	Référence francophone : tubes, redirections, shell.
Guide ANSSI, *Règles de programmation pour le développement sécurisé de logiciels en langage C*	Vérification systématique des retours, validation des entrées.

Pour approfondir

CULTURE **Les autres IPC**

Les tubes ne sont qu'un des mécanismes de communication entre processus. Les autres : **signaux** (`kill`, `sigaction`), **files de messages** et **mémoire partagée** POSIX (`mq_open`, `shm_open`), **sockets du domaine UNIX** (bidirectionnelles, avec envoi de descripteurs), et les **sockets réseau** du sujet 09. Une socket du domaine UNIX se programme comme une socket TCP, mais reste locale à la machine.

CULTURE **Pourquoi « Broken pipe » ?**

Dans `yes | head -1`, `head` se termine après une ligne ; `yes` écrit alors dans un tube sans lecteur et reçoit `SIGPIPE`, qui le tue silencieusement. C'est voulu : sans ce signal, `yes` tournerait à l'infini en écrivant dans le vide. Le shell ne signale pas cette mort, car elle est normale dans un pipeline.

CULTURE **Tubes et sécurité des privilèges**

Un programme privilégié qui exécute un autre programme lui transmet ses descripteurs ouverts : c'est une source classique de fuites (fichier de mots de passe, socket d'administration). C'est pourquoi les API modernes créent les descripteurs avec close-on-exec par défaut (`pipe2(p, O_CLOEXEC)`, `open(..., O_CLOEXEC)`, `SOCK_CLOEXEC`).